High-Performance Computer Architectures
Practical Course

Script

High-Performance Computing
Prof. Dr. Ivan Kisel
Robin Lakos
Akhil Mithran
Oddharak Tyagi

October 22, 2024

Dear students,

Welcome to the High-Performance Computer Architectures practical course. The goal of this course is to provide you an introduction to software development for high-performance computers including simplified applications from our daily research routine. The modern low-level programming language C++ allows to develop software close to the hardware specifications and allows to program runtime- and memory-efficient applications, running on CPU and GPU. Within this course, you will learn how to work with Unix-based systems and C++ to write smaller applications that make use of several concepts of parallel programming such as multi-threading and vectorization.

If you have any questions regarding the examples given, please ask your tutor.

Please note: During the course, we will add new sections and provide more examples for the specific topics to provide a good overview of the course requirements. If you have any suggestions or wishes that could help you, please let us know.

Contents

Chapter 1

Introduction - Unix Shell and C++

1.1 Unix Shell and Linux

1.1.1 Unix History and High-Performance Computing

Without going into too much detail, Unix is a family of multi-tasking, multi-user operating systems developed in 1969. Nowadays, there are several operating systems based on it, including Linux distributions and macOS.

Although many people are probably more familiar with Windows as an operating system, many other devices run on Linux distributions (Linux-based operating systems), including high-performance computer farms and many servers. Linux itself is a kernel, and therefore, it builds the interface between the system hardware and the operating system. Its open source nature makes it suitable for research and high-performance computing: anyone can investigate the underlying source code, many people around the world search for bugs, performance issues or security problems, and it can be modified to the own needs.

In general, there are many Linux based operating systems. On the one hand, distributions such as Debian and Fedora are known as rock-stable operating systems, why they are often preferred for servers, where stability is crucial. On the other hand, these distributions are sometimes not considered beginner friendly nor up-to-date on cutting-edge software. For each of these cases there are nowadays different Linux distributions, often developed by communities and sometimes financial supported by or developed directly by larger companies. When it comes to beginner friendliness, it is also helpful to have a look at different Desktop Environments (DEs) that are available for the distributions.

1.1.2 Terminal and Shell Commands

All C++ applications of this course will be compiled and started via terminal/console, using the Unix Shell. This allows precise instructions and allows to avoid overhead by a graphical user interface and other software requirements. The terminal is a powerful tool to work on a computer. However, starting with it can feel slow and complex. In this part, a few basics are discussed as a preparation for the following exercises. Starting a terminal on Linux, it is likely to prompt `username@computername` followed by a *path*, the current *working directory* (depending on the terminal application or theme, this might differ). Starting the terminal application, the current working directory is usually the user's home directory.

One can use the **pwd** command to **p**rint **w**orking **d**irectory. This command returns the absolute path to the directory the user is currently in (via terminal) and prints it to the console. All directories are separated by **/**. The root directory, however, is an exception, since it is also named just **/**.

To change to a different directory, one can use **cd** for **c**hange **d**irectory with a given path. The path can be an absolute path, starting from the root directory, or a relative path, starting from the current position. The `ls` command can be use to **l**i**s**t the contents of a directory.

```
username@hostname:~$ pwd
/home/hpc
username@hostname:~$ ls
directoryA directoryB directoryC
username@hostname:~$ cd directoryA
username@hostname:~/directoryA$ cd directory-1/directory-1-1
username@hostname:~/directoryA/directory-1/directory-1-1$
username@hostname:~/directoryA/directory-1/directory-1-1$ cd ../../..
username@hostname:~$
```

When the user has to change between two directories that have both long paths, **cd -** is a good shortcut to avoid the long paths multiple times. It can be used switch to the lastly used working directory.

```
username@hostname:~$ pwd
/home/hpc
username@hostname:~$ cd directoryA/directory-1/directory-1-1
username@hostname:~/directoryA/directory-1/directory-1-1$ DO SOMETHING HERE
username@hostname:~/directoryA/directory-1/directory-1-1$ cd
↪  ../../../directoryB/directory-3/directory-2-1
username@hostname:~/directoryB/directory-3/directory-2-1$ DO SOMETHING THERE
username@hostname:~/directoryB/directory-3/directory-2-1$ cd -
username@hostname:~/directoryA/directory-1/directory-1-1$ DO SOMETHING HERE
↪  AGAIN
```

During this course, try to learn to work with the shell. For high-performance computing, it is a crucial tool to work with.

1.2 C++ Basics

1.2.1 The History and Path of C++

C++ was created by Bjarne Stroustrup, as an extension to the C language, in 1985. One of the main differences between C and C++ is that C++ supports classes and objects, while C does not. Due to the existence of classes, it is also referred to as an object-oriented programming language. Compared to low-level languages like Assembly, C++ can be considered as a high-level language. In a very basic sense, high-level languages are those that require only a few lines of code to

execute a large number of tasks and are also easily understandable for humans, given their syntax. However, when compared to newer languages like Python, some might group C and C++ as a low-level language, since they allow to program close to hardware, which makes it very efficient.

C++ is a growing language in that new standards for its specifications and features are released every three years, starting from 2012, by the International Organization for Standardization (ISO). As usual in programming courses, not all topics of C++ can be covered, although the most important are. Therefore, below you will find good references for C++ that help in case you need more information.

Online resources:
(1) C++ language and library reference:

- https://cppreference.com/
- somewhat formal, but precise, with examples

(2) C++ core guidelines:

- https://isocpp.github.io/CppCoreGuidelines/
- how to use C++ correctly

1.2.2 Repetition: Programming Languages

Before starting with C++, some basic concepts of programming are repeated in this section.

Definitions:

- A *computer* is a device that executes programs.
- A *program* is a precise sequence of instructions, enabling a computer to perform a specific task.

A computer as we know it only ”understands” binary, as it is build from electronic parts that simply work based on present or not present voltage/current. Therefore, simplified, the signals send through a computer can only assume two states (0 and 1). In digital computers, thresholds are used to make these states binary, which helps to reduce slight fluctuations and noise due to the physics behind and allows to apply boolean logic.

The exact language a computer ”speaks” is based on the specific hardware (architecture). For example, an ARM-based processor speaks a different language than a x86_64-based processor. Thus, the data used to perform specific tasks has to be in binary as well. However, the interpretation of the bits and bytes is additionally depending on the defined *data type*:

`01100101 01110110 01101001 01101100` read as float (IEEE-754 32-Bit) is `7.272793e22`

`01100101 01110110 01101001 01101100` read as int (32-Bit) is `1702259052`

`01100101 01110110 01101001 01101100` read as ASCII/UTF-8 is `"evil"`

A (high-level) programming language is an artificial language developed for humans to give precisely defined instructions to a computer that are still close to human languages. Human languages have interpretative flexibility, while programming languages are designed with a specific syntax and semantics, meaning each instruction is (usually) strictly defined and performs a particular operation as dictated by the language's rules.

Since a computer only "speaks" binary, these instructions have to be translated into binary, which is usually done by a compiler or an interpreter. The compiler or interpreter itself is a program that translates from the programming language into machine readable instructions for the underlying architecture. While a compiler takes all of the source code for translation at once to create an executable program application, an interpreter reads and translates the source code line by line.

The result of the compilation (the program in binary language) is usually called *executable*. When moving the executable to another computer that speaks a different language (e.g. it has a different architecture), it has to be re-compiled. After the executable is created, the source code is not necessary anymore to run the application on the same architecture.

C++ is almost[1] always implemented as a compiled language, and many vendors provide C++ compilers, including the Free Software Foundation, LLVM, Microsoft, Intel, Embarcadero, Oracle, and IBM. The compilers suggested for this course are either the `g++` compiler by Free Software Foundation or `clang++` compiler by LLVM.

1.2.3 Compiling C++ Source Code - Single Source File

As already mentioned, the compilation of source code is the translation from a programming language into machine instructions. For a single file, this process is more intuitive and, thus, we start with a simple single source code file. Later on, more complex projects are discussed in more detail.

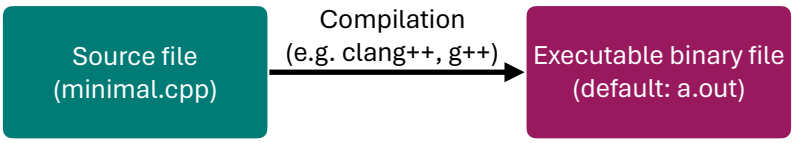

Figure 1.1: Translation of C++ source code into executable machine instructions.

First, let's create the obligatory "Hello world!" example. To write the first C++ code, we create a plain text file with `.cpp` ending, named `HelloWorld.cpp`, and add the following content:

```
#include <iostream>

int main(int agrc, char** argv)
{
    std::cout << "Hello world!" << std::endl;
    return 0;
}
```

[1] There are actually implementations of Just-In-Time (JIT) compilers for C++ that behave to some sense similar to an interpreter. One famous example is the ROOT Framework developed at CERN that contains a C++ JIT compiler.

The `include`-statement includes the iostream library, which provides facilities for input and output through streams like std::cout for output and std::cin for input. Afterwards we define the `main` function, which is the entry point of a C++ program. The number of command-line arguments passed to the program is represented by `argc`, which includes the program name itself. The second function parameter, `argv`, is a pointer to the first element of an array of pointers to characters, essentially representing all the command-line arguments passed to the program. Inside the function body, there are two lines. The first of these lines uses the `std::cout` stream to output the string "Hello world!" to the console. The « operator is used to send data into the stream. When data, e.g. a string, is passed into the `std::cout` function, it returns the `std::cout` function. That way, `std::endl` is also streamed into `std::cout`, used to insert a newline character into the stream and flush the output buffer, ensuring that the output is immediately visible. The second line returns 0 from the `main` function, signaling to the operating system that the program has executed successfully. Returning a non-zero value typically indicates an error.

In the following example, the source code file `HelloWorld.cpp` is compiled into an executable output file `HelloWorld.out` (line 1). Then, the application can be launched by running the executable file (line 2), such that the output is created (line 3), and here, printed to console.

```
1 username@hostname:~$ clang++ HelloWorld.cpp -o HelloWorld.out
2 username@hostname:~$ ./HelloWorld.out
3 Hello world!
```

1.2.4 Primitive/Fundamental Data Types in C++

There are several *primitive* or *fundamental data types* in C++ that help programmers to build more complex structures and data types, such as:

Data Type	(Typical) Size in Bits
`bool`	1
`char`	8
`int`	32
`long`	64
`float`	32
`double`	64

Table 1.1: Example of a few fundamental data types and their typical sizes on modern PCs.

Here, these data types are given with their typical sizes in bytes. In the C++ standard, sizes of fundamental data types are given as minimum required precision, not as a fixed size value. For example, the data type `int` requires at least 16 Bit / 2 Bytes, but usually comes with 32 Bit / 4 Bytes on modern computer architectures. More on that here: https://en.cppreference.com/w/cpp/language/types.

This decision is rooted historically and is influenced by the need for portability and compatibility (e.g. backward compatibility) across various platforms, including those with different word sizes and memory constraints, e.g. in embedded systems.
These sizes are sometimes neglected as long as the used data types are sufficiently large to store the data. In high-performance computing, however, these can have crucial factors with respect to

the runtime. For example, the charge of a particle in heavy-ion physics experiments can have one of three different states: positively (+1), neutrally (0) or negatively (-1) charged. Therefore, a datatype that can store at least three different states is sufficient.
A bool with only two states (true, false) is not sufficient and the next largest integer type that can represent it is a `short` with 16 Bit on modern computers. Intuitively, this would make sense as it can easily store negative and positive numbers. However, a `char` has a size of 8 Bits and can also easily represent the three states. Although it is a character type, it can be used to store the charge of a particle. For each particle, there are 2 times less Bits required to store its electrical charge when using `char` in comparison to `short`.

But does it make a difference? Well, it depends. If the computer reads a line of aligned memory (RAM) and stores it into the cache, twice as much values can be stored when we would store the charges of particles as `char`. Access to the memory is quite costly, that sometimes even re-calculation can be much faster than reading it from memory. Just to provide some numbers: a memory access can easily take up to 100 ns, whereas L1 cache access can be as fast as 0.5 ns.
Nevertheless, to achieve an efficient runtime, optimal data types help definitely, but how they are stored and aligned in memory is at least as crucial as finding the best matching one. Later in this course, different types of storing data are discussed in more detail.

1.2.5 Types and Casting

C++ is a statically and strongly typed language, which means the types of variables must be known at compile-time. This attribute enforces type safety by generating errors if variable types do not match during compilation, limiting implicit conversions. It is imperative in C++ that every variable and function return type is explicitly declared. In C++ 11, the *auto* keyword was introduced, enabling the compiler to infer variable types at compile time, enhancing code readability and maintainability, especially in complex scenarios like iterating over containers or using lambdas.

Regarding type casting, C++ supports conversions between different types, categorized into implicit and explicit conversions. Implicit conversions are automatically performed by the compiler, whereas explicit conversions require specific syntax to instruct the compiler. Although C++ maintains backward compatibility with C-style casting, it is advisable to utilize C++'s four cast operators for clearer, type-safe conversions. This recommendation stems from the fact that C-style casting can allow conversions between incompatible types, posing risks for code maintenance and clarity, especially in collaborative environments.

The C++ cast operators are:

- `static_cast<T>`: Suitable for non-polymorphic type conversions, including upcasting and downcasting within class hierarchies, numeric type conversions, and pointer type conversions where implicit or reversible. This operator ensures type safety through compile-time checks.

- `dynamic_cast<T>`: Utilized for safe downcasting in class hierarchies, converting a base class pointer or reference to a derived class pointer or reference. It employs Run-Time Type Information (RTTI) to verify the cast's validity, returning a null pointer or throwing a `bad_cast` exception upon failure.

- `reinterpret_cast<T>`: Enables conversions between pointer types or between pointer and integer types, performing a bit-wise copy of the value. Its use is generally discouraged due to potential alignment and type punning issues, and should only be considered when absolutely necessary.

- `const_cast<T>`: The only cast capable of modifying the constness of an object, useful for invoking non-const methods on const objects or modifying objects initially declared as const. This operator can add or remove the `const`, `volatile`, or `__unaligned` qualifier from a pointer or reference.

Transitioning from the basics of types and casting, let's delve into practical examples to illustrate these concepts further.

Examples:

```
// different static type-casting variants
short a = 10;
float b;
b = a;                      // implicit (variant 0)
b = (float) a;              // explicit (variant 1)
b = float(a);               // explicit (variant 2)
b = static_cast<float>(a); // explicit (variant 3)

// dynamic type-casting for pointers/references to objects
Animal* animal = new Dog();
Dog* dog = dynamic_cast<Dog*>(animal);

// reinterpreting memory, here array interpreted as __m128 vector:
float data[] = {1.f, 2.f, 3.f, 4.f};
__m128 &vec = reinterpret_cast<__m128&>(data);

// modifying constness of an object to re-write const-objects:
int c = 10;
const int& d = c;           // create "write-protected" reference
const_cast<int&>(d) = 5;    // re-write "write-protected" reference
```

1.2.6 Variables, Literals, and Expressions

In C++, *variables* are identifiers providing a name to objects in memory, for instance, `int a;` defines a variable named `a` of type `int`. *Literals* represent constant values directly included in the code, like `5` in `a = 5;`. An *expression* involves operators and operands performing computations, such as `a++`, which increments `a`.

Each object in C++ is characterized not just by its data type and size, but also by two main properties:

– An address, indicating the memory location of the object.

– Content, the data value stored at the object's memory location.

```cpp
int a = 42; // Creates an integer variable named 'a' with a value of 42
a = 1337;   // Modifies the content of 'a' to 1337
```

1.2.7 Initialization in C++

C++ supports various initialization methods. Traditional forms include:

- Direct Initialization: `int a(5);`
- Copy Initialization: `int a = 5;`

C++11 introduced *Brace Initialization*, offering advantages such as a uniform syntax across all types, prevention of narrowing conversions, and uniform initialization of aggregates (arrays, structs):

```cpp
int a{4};     // No error
int b{7.9};   // Compiler error due to narrowing
```

Initialization of data structures:

```cpp
std::vector<int> v{1, 2, 3};
int array[]{1, 2, 3};
YourClass obj{};
```

This initialization method enhances code safety and is thus recommended in modern C++ practices.

1.2.8 Pointers in C++

Your introduction effectively sets the stage for understanding memory addresses. Clarifying that every memory cell can be addressed and that variables are allocated to these addresses could be beneficial. It might be helpful to mention that the size and alignment of variables affect how they are placed in memory, which is managed by the compiler and the runtime environment.
Any variables in C++ can be accessed with its identifier (name). This is the easiest way when we don't need to care about the physical location of our data in the memory. However, there is a second way to access a variable, directly using its location in the memory. The computer's memory can be represented as a sequence of memory cells of one byte each. These cells are numbered in a consecutive way. Each cell has a unique number in the whole available memory. Every next cell has the number of the previous one plus one. This way we can claim that the cell number 55 definitely follows the cell number 54. As soon as we declare a variable, the amount it needs in memory is allocated in a specific location (memory address). The operating system performs this task automatically during runtime. This memory address locates a variable in the memory. This memory address can be obtained by adding an ampersand sign (`&`), the so-called *address operator*, in front of the identifier. So `&a` gets the address of the variable a. A variable

which stores the memory address of another variable is called a pointer.

Pointers are a foundational aspect of C++, allowing variables to reference memory locations of objects. Key points regarding pointers include:

- Pointers have a consistent size, typically making them more efficient to move or copy compared to large data structures.
- Modifying an object via a pointer affects the original object globally.
- Pointers facilitate data structure manipulations, such as swapping nodes in trees without moving all child nodes.

Pointers are said to ”point to” the variable whose address they store. All pointers have a special pointer type. While declaring a pointer, one has to specify this type. The pointer type is obtained by adding an asterisk after the type it points to.

```
int a = 5;    // 5 stored at some memory location, let's say at 0x7ffcdf8a990c
int* b = &a;  // 'b' points to the address of 'a' (0x7ffcdf8a990c)
              // 'b' itself is located somewhere else, e.g. at 0x7ffcdf8a9900
```

The content of a pointer is a memory address and the address where the pointer is pointing to has some content, which can be accessed by dereferencing the pointer.

Dereferencing Pointers: The value (content) at the pointer's referenced memory location can be accessed by dereferencing the pointer. One can use the asterik sign (*) in front of the pointer variable to dereference the pointer. In this case, the * is called *dereference operator*.

```
std::cout « ”address of a: ” « &a « ”, content of b: ” « b « std::endl;
std::cout « ”content of a: ” « a « ”, dereferenced b: ” « *b « std::endl;
std::cout « ”address of b: ” « &b « std::endl;
```

Additionally, operations can be applied to pointers, like arithmetic operations to traverse arrays.

```
int arr[2];
int* p = arr; // same as ”int* p = &arr[0]” -> p points to the first element
++p;          // p points now to the second element
```

In this example, the behavior of `arr` is similar to pointer: one can either set `p` to `arr` or to the address of element 0 of the array, but not to the address of `arr`. It might seem like the content of `arr` is an address, similar to pointers, but actually `arr` is a array-type. But arrays in expressions decay to pointers to their first element, reflecting a design choice from C for simplicity and efficiency.
This conversion facilitates passing arrays to functions as pointers, enabling efficient data manipulation. However, a critical implication of array decay is the loss of size information. When an array decays to a pointer, the function receiving the pointer does not inherently know the size of the original array. This necessitates careful management, often requiring the size of the array to

be passed as an additional parameter to functions that operate on the array. Understanding and accounting for array decay is essential to avoid out-of-bounds access, ensuring safe and effective use of arrays in C++. In modern C++, other container classes/structs are usually used, such as `std::vector<T>` or `std::array<T>`.

Fundamentally, a pointer is designed to hold the address of a single memory location. However, when it references an object occupying more than one byte in memory – such as a multi-byte data type or an array – the pointer is actually pointing to the initial byte of a contiguous block of memory where the object resides. For instance, a pointer referencing a 4-byte integer, or an array thereof, essentially points to the first byte of this integer. When such a pointer is incremented, it advances to the next element in the sequence. The magnitude of this advancement, or ”jump”, is determined by the size of the object type to which the pointer refers. Consequently, for an array of 4-byte integers, incrementing the pointer results in it moving 4 bytes forward, aligning with the start of the subsequent element in the array. Thus, pointer-arithmetic has type-awareness.

Please note: The `this` keyword in C++, representing the object itself, is also a pointer.

1.2.9 References in C++

References in C++ create an alias for an existing variable, serving as a powerful tool in high-performance computing by circumventing the need for costly copies. They find their primary utility in two scenarios: as return types and as function parameters, significantly enhancing performance by operating directly on the original objects.

Consider the distinction between working with a copy versus a reference:

Working with a copy:

```
Matrix matrix = obj.GetLargeMatrix(); // Incurs significant overhead by copying
matrix.DoSomething();
```

Working with a reference:

```
Matrix& matrix = obj.GetLargeMatrix(); // Efficiently accesses the original
matrix.DoSomething();
```

To utilize a reference, one has to change the function’s return type to return a specific reference type, e.g. `Matrix& GetLargeMatrix()`.

This technique is similarly applied to function parameters to ensure that operations are performed on the original data without incurring the overhead of copying. Function parameters declared with an &-sign are treated as references, enabling direct manipulation of arguments as if they were the originals. This approach is not only efficient but also maintains code clarity and intuitive logic flow. However, it is crucial to ensure that returned references do not point to local objects within a function scope to avoid dangling references and the errors resulting from this.

```
Matrix MatrixMultiplication(const Matrix& a, const Matrix& b)
```

Here, two references of type `Matrix` are expected as function parameters. In a non-static function that is applied directly to an object, one matrix (e.g. Matrix 'a') could be the object the function is applied to and then only one matrix (e.g. Matrix 'b') would be given as a parameter.
However, this is somewhat a design choice. In some cases, in high-performance computing, you will also see that the result reference is given as a function parameter. For example, when memory is already pre-allocated for re-using it multiple times, such a concept makes sense and avoids multiple re-allocations of large spaces in memory. In the end, this is usually a trade-off between intuitive code and performance. For maintenance reasons, you should not always opt for performance, but in limited environments, it might help.
The return type here is not a reference, as in this example, it is assumed that the result is a new matrix and not one of the two operands is overwritten. When creating objects inside a function body that is then not stored in another object with a longer lifetime, one can not return a reference to that object. When the function body (scope) ends, the lifetime of the new object ends and the reference would be referring to an object that does not exist anymore, leading to an error.
Additionally, in the example above, the `const` keyword is used to avoid accidental modifications to the matrices. The keyword will be discussed in more detail later on.

A reference is implemented similarly to a pointer (e.g. it has a small size of 8 bytes in 64-Bit systems), while a copy is always depending on the target object's size.

When working with large data structures, copying a large object is timely expensive and should be avoided whenever possible.

1. Memory has to be allocated for the new object
2. The whole object has to be copied into the new location

Avoiding unnecessary copies not only saves time but also reduces memory usage, which is critical in high-performance computing and memory-constrained environments like embedded systems.

1.2.10 Const Correctness

Ensuring `const` correctness in function parameters and return types is a best practice in C++ programming. This involves using `const` with references to prevent modification of the original object, enhancing code safety and expressiveness.

Example:

```
1 void DisplayMatrix(const Matrix& matrix) {
2     // Display the matrix without modifying it
3 }
```

Applying `const` to reference parameters enforces immutability, allowing read-only access to the object. This guarantees that the function will not alter the object's state, providing clarity to other developers and preventing unintended side effects.
Furthermore, `const` correctness aids in the optimization of compiler-generated code, potentially unlocking performance enhancements through the assurance that certain objects remain unchanged throughout their use. This can also have a non-negligible effect on the runtime.

Const References as Return Types

While returning a reference can significantly optimize performance by avoiding unnecessary copies, combining it with `const` ensures the returned object is protected against unintended modifications. This technique is particularly useful when returning objects that are members of a class or have a static lifetime, ensuring the safety and integrity of the returned reference.

Example:

```
const Matrix& GetReadOnlyMatrix() {
    static Matrix matrix;
    // Perform operations
return matrix; // Returns a read-only reference
}
```

Incorporating `const` correctness into your C++ programming practices promotes not only efficiency and resource conservation but also contributes to the development of robust, error-resistant code.

1.2.11 Control Flow

C++ allows the programmer to have different paths of execution through the program. This is achieved via a collection of control flow statements. The simplest example is perhaps the if-statement that lets us execute a statement only if a conditional expression is true. For ease of understanding we will divide control flow statements into six categories: conditional statements, jumps, function calls, loops, halts and exceptions.

Conditional statements cause a block of code to be executed if and only if a certain condition, such as an equality (==) or inequality (>,<, <= etc.), is satisfied. The if-else and switch statements fall into this category. Jump statements tell the CPU to execute statements at some other location in the program. The break, continue and goto statements are jump statements. Loops tell the program to repeatedly execute some sequence until some condition is met. The while, do-while, for and ranged-for statements are used to generate loops. Exceptions are a special kind of statements designed for error handling such as unexpected user input. C++ uses try, throw and catch for exception handling. You will not encounter many uses of exceptions in this course, but the exception handling is important generally. Function calls are jumps to some other location in the code and back. Function calls and return are used together to achieve this effect. Finally, In some situations the only recourse is to terminate the program. Halt statements like std::exit() and std::abort() statements can be used for this.

C++ allows the programmer to nest control flow statements i.e. it is allowed to use, for example, loops inside loops which may themselves be inside of an if statement. This nesting of control flow statements allows the program to take highly convoluted and input dependent paths. A good command over control flow statements is absolutely necessary to follow the rest of this course. Please spend time to master these statements.

The following example provides at least some applications of conditional statements and loops.

```
int a = 5;
int b = 10;
int c = (a > b) ? b : a--;

for (int i = 0; i < c; i++) {
    do {
        std::cout « "a: " « a « std::endl;
        a--;
    }
    while (a > -1);

    std::cout « "i: " « i « std::endl;
}
```

1.2.12 Stack vs. Heap

There are primarily two types of memory allocations in C++: stack and heap allocations. For stack allocation, the allocation happens on contiguous blocks of memory. We call it a stack memory allocation because the allocation happens in the function call stack. The size of memory to be allocated is known to the compiler and whenever a function is called, its variables get memory allocated on the stack. And whenever the function call is over, the memory for the variables is deallocated. A programmer does not have to worry about memory allocation and deallocation of stack variables. In the case of heap, the memory is allocated during the execution of instructions written by programmers. Every time when we make an object using the operators for dynamic allocation, it is always created in heap-space and the referencing information to these objects is always stored in stack-memory. Heap memory allocation is not as safe as Stack memory allocation because the data stored in this space is accessible or visible to all threads. If a programmer does not handle this memory well, a memory leak can happen in the program.

```
int a = 5;              // int allocated on stack
int* b = new int(3);    // int allocated on heap

// print out "5 3":
std::cout « a « " " « *b « std::endl;

delete b;               // free heap-allocated memory manually
```

1.2.13 Classes and Structs

Classes and structs are the constructs whereby the user can define their own types. The two constructs are identical in C++ except that in structs the default accessibility is public, whereas in classes the default is private. The term accessibility here refers to the allowed access to the members of the class/struct. Access controls enable you to separate the public interface of a class, which can be interacted with "directly" after an object from the class is created, from the private

implementation details, which are set during object creation and changed using member functions of the class, and the protected members that are only for use by derived classes. The access specifier applies to all members declared after it until the next access specifier is encountered. Classes and structs can both contain data members and member functions, which enable you to describe the type's state and behavior. Structs, however, are often used to create small data structures, whereas classes are often used for more complex objects.

```
struct Node {
    int id;
    int value;
};

struct Edge {
    Node* u;
    Node* v;
};


class Graph {
private:
    int nNodes_;
    int nEdges_;

    Node* nodes_;
    Edge* edges_;
protected:
    // member functions and variables
public:
    Graph(Node* nodes, Edge* edges, int nNodes, int nEdges) :
        nodes_(nodes),
        edges_(edges),
        nNodes_(nNodes),
        nEdges_(nEdges)
    {
        // other constructor code
    }

    // member functions and variables
};
```

When working with classes in multiple header and source files, it is important to set guards to ensure that every class is included once. This can be done by the following guards:

```
#ifndef CLASSNAME_H
#define CLASSNAME_H

class CLASSNAME
```

```
{
    // class code here
};

#endif // CLASSNAME_H
```

1.2.14 Templates

The template system is designed to simplify the process of creating functions (or classes) that are able to work with different data types. Instead of manually writing a bunch of almost identical functions or classes (one for each set of different types), we can instead create a single template. Just like a normal definition, a template describes what a function or class looks like. However unlike a normal definition (where all types must be specified), a template uses one or more placeholder types. A placeholder type represents some type that is not known at the time the template is written, but that will be provided later. Once a template is defined, the compiler can use the template to generate as many overloaded functions (or classes) as needed, each using different actual types! Thus we only have to create and maintain a single template, and the compiler does all the hard work by replacing the types respectively. In fact, the C++ standard library itself is absolutely full of template code.

To create a template function, replace instances of specific types (such as int or float) with type template parameters which we name, for instance, T. Now we have to declare the template parameter to let the compiler know that the function is a template. We do this by adding the following line before the function declaration: template <typename T>.

```
// function template
template <typename T>
T GetMax (T a, T b) {
    if (a > b) return a;
    return b;
}

// class template
template <class T>
class Pair {
private:
    T a, b;

public:
    Pair (T first, T second) : a(first), b(second)
    {
        // other constructor code
    }

    T GetMax()
    {
```

```
22        if (a > b) return a;
23        return b;
24    }
25 }
```

1.2.15 Multi-File Projects and (Meta-)Build-Tools

Many projects consist of more than a single source code file. There are several tools that can be used to handle projects. A well-known tool is CMake, that allows to create a configuration file with all relevant information, such as files to compile, compiler settings, dependency checks and much more. In the following example, we assume a project in the `project` directory. In this directory, there is the CMake configuration `CMakeList.txt` that contains all relevant information required for compiling the application. Depending on the project structure, there are code files or other directories with the code included that are then loaded by the `CMakeList.txt`.
To compile our project, a `build` directory is created using the "make directory"-command `mkdir`. A second step is to navigate into this directory using "change directory"-command `cd`. Then we use the CMake tool to setup our code for compilation, using the `CMakeList.txt` in the directory outside of the build directory - CMake will automatically search for it. After this, we can run `make` to build our project and run our application.

```
username@hostname:~/project$ mkdir build
username@hostname:~/project$ cd build
username@hostname:~/project/build$ cmake ../.
username@hostname:~/project/build$ make
username@hostname:~/project/build$ ./PROJECTNAME
```

A simple `CMakeLists.txt` file could look like this:

```
cmake_minimum_required(VERSION 3.22)
project(PROJECTNAME)

set(CMAKE_CXX_STANDARD 14)

include_directories(.)

add_executable(PROJECTNAME
    Project.cpp
    ClassA.h
    ClassA.cpp
    ClassB.h
    ClassC.cpp)
```